Any App Is a Key: A Software–Hardware Collaborative AI System Achieving Imperceptible Full-Lineup Vehicle Control Attacks Against Tesla

Xinfeng Chen
Geely Zero Security Lab
Zhejiang Geely Automobile Research Institute Co., Ltd.)
Ningbo, China
Xinfeng.Chen2@geely.com

Mingming Wan
Geely Zero Security Lab
Zhejiang Geely Automobile Research Institute Co., Ltd.)
Ningbo, China
Mingming.Wan1@geely.com

Wenjie Chen
Geely Zero Security Lab
Zhejiang Geely Automobile Research Institute Co., Ltd.)
Ningbo, China
Wenjie.Chen21@geely.com

***Abstract*—Your phone has the Tesla App installed, and your car is parked outside—an AI agent-driven attack chain allows any app on your phone to open and unlock your vehicle. This paper discloses three new vulnerabilities in the Tesla App digital key ecosystem. One of them affects the entire vehicle lineup, allowing an application that requests no permissions whatsoever to directly control the vehicle—with the owner completely unaware throughout the process (CVE-2026-110446, acknowledged and fixed by Tesla, Bugcrowd bounty USD 1,500). Another vulnerability can silently rewrite the navigation destination of the in-vehicle infotainment system; the vendor declined to fix it on the grounds of "intended design"—we publicly release the complete attack chain and real-vehicle footage for independent judgment by the community. The third vulnerability was deemed by the vendor to be a duplicate of an existing external report and was not assigned a separate identifier. In addition, this method has collectively yielded 3 CVEs and 22 China CAPPVD identifiers across multiple leading automakers.**

Supporting all of this is AutoAudit-X (AX)—the first AI Agent team that combines software and hardware: the Agent team is responsible for reasoning and adversarial cross-review (reconnaissance, auditing, and heterogeneous-model cross-validation to suppress false negatives); the system and hardware substrate is responsible for observation (Root hiding, unpacking, method-level hooking, and TLS key extraction are compiled directly into the Android system, while a dual-track BLE research bench extends the attack surface from the App layer to the physical vehicle). When the strongest publicly available AI Agent is helpless against a hardened automaker App, AutoAudit-X runs the entire attack chain all the way to the vehicle side.

Core thesis: The next step for AI security is not merely optimizing Agents, but rebuilding a system- and hardware-level observation substrate for Agents.

Keywords— Application Security: Offense; AI & ML: Attack and Defens)

I. INTRODUCTION

A. A Problem Too Hard for Both Humans and AI

The attack surface of the Internet of Vehicles (IoV) is distributed across three heterogeneous layers: the mobile App is the core entry point for vehicle control and harbors conventional mobile security weaknesses; cloud services carry authentication and business interfaces and are high-value targets for attackers; and the BLE digital key between the App and the vehicle extends the attack chain from the digital world to the physical vehicle.

The attack chain itself requires traversing all three layers: an entry point discovered at the App layer may amplify its impact through cloud services and ultimately materialize as physical control over a real vehicle via a BLE protocol flaw. Yet the existing toolchain has a breakpoint at every layer:

- The App layer is opaque: commercial hardening (instruction extraction, VMP-ization, anti-debugging, environment self-checks) renders conventional dynamic analysis tools ineffective across all three dimensions—injection timing, detection countermeasures, and stability; even if the hardening is bypassed, Android 7+ does not trust user CAs by default, and certificate pinning reduces man-in-the-middle interception to nothing but handshake failures.
- The cloud layer is illegible: business traffic layers multiple custom encryption schemes on top of TLS, so packet capture alone cannot recover protocol semantics; server-side authentication and privilege-escalation auditing require full client-side context.
- The vehicle side is out of reach: BLE digital key protocol analysis requires specialized hardware and a real vehicle, and protocol implementations vary widely—manual analysis of a single vehicle takes weeks.

Failure of observation at any one link renders the entire attack chain unverifiable.

B. *The Observation Blind Spots of AI Agents*

In 2026, AI Agents have moved from concept demonstrations to real-world offense and defense: OpenAI disclosed at Black Hat USA 2026 that multiple Agents in its internal evaluations autonomously discovered 0days and laterally moved into production environments; GitHub Security Lab's seclab-taskflows used LLM Agents to uncover 24 vulnerabilities in Android applications.

Yet all of these results rest on an implicit premise—that the target is observable: the target is not hardened, traffic can be decrypted, and verification can be completed in a software environment. Faced with a cross-layer, cross-domain, physics-involving target such as the Internet of Vehicles, the capabilities of AI Agents cannot be brought to bear.

Core thesis: When the target is wrapped layer upon layer in commercial hardening, multi-layer encryption, and proprietary protocols, the bottleneck lies not in intelligence but in observation.

C. *Our Answer*

We propose AutoAudit-X (AX)—an AI4Sec framework that deeply integrates a multi-Agent collaborative system with a system-endogenous observation engine, HookLogic-X. Unlike existing work that makes localized improvements within individual layers, AX integrates three categories of capability within a single framework:

- Observation capability is undertaken by HookLogic-X—unpacking, method-level hooking, and key-extraction primitives are implanted directly into the runtime substrate through AOSP precompilation modifications, and are exposed upward to upper layers as system interfaces;
- Reasoning capability is undertaken by the multi-Agent collaborative system—reconnaissance, auditing, and adversarial cross-review are executed asynchronously by specialized agents and cross-validated;
- Physical verification capability is undertaken by the dual-track BLE research bench—protocol capture and attack interaction are closed-loop on real vehicles.

The underlying observation power thus empowers the upper-layer intelligent agents: Agents invoke system interfaces at runtime and drive real-device and real-vehicle interaction, forming an end-to-end audit loop from attack-surface exploration to PoC output, and from hardened Apps to physical vehicles.

II. Core Disclosures: Three Tesla Vulnerabilities

A. *Vulnerability 1*

Imperceptible Vehicle Control by a Zero-Permission App (CVE-2026-110446)

A third-party App that requests no permissions whatsoever can directly control the owner's vehicle—with the owner completely unaware throughout the process.

Scope of impact: Affects the entire vehicle lineup.

Vendor response: Tesla has acknowledged the vulnerability and completed a security fix, and has issued a USD 1,500 bounty through the Bugcrowd platform. The complete attack chain and real-vehicle verification details will be presented at the conference.

B. *Vulnerability 2*

Imperceptible Tampering with the In-Vehicle Navigation Destination (Vendor Refuses to Fix)

A complete report and reproduction steps were submitted through Tesla's official vulnerability disclosure channel; the vendor responded by deeming it "intended design behavior" and declined to fix it. We will publicly release the complete attack chain and real-vehicle test footage on site, present a counter-evidence chain, and leave the matter to the community for independent judgment.

C. *Vulnerability 3*

Duplicate Report (No Separate Identifier Assigned)

The vendor deemed it a duplicate of an existing external report, and no separate identifier was assigned; a brief explanation is provided.

III. Vulnerability Discovery Methodology: The AutoAudit-X Technical System

The overall architecture of AX is divided into four layers—the Agent layer, the ROM layer, the collaboration zone, and the hardware layer:

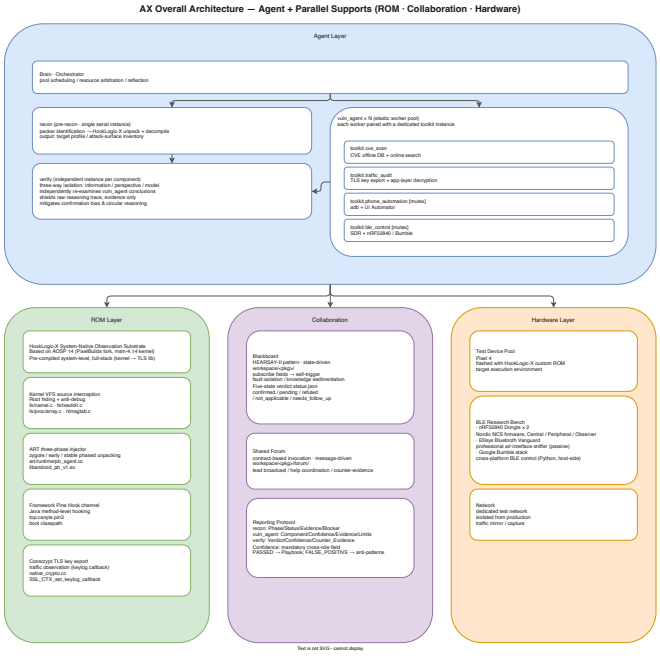


Fig. 1 Overall architecture of AutoAudit-X. The Agent layer consists of ROM, Collaboration Zone, and Hardware Support.

Four-layer collaboration:

- Agent layer: a three-agent architecture (recon / vuln_agent / verify) plus a toolkit layer, uniformly scheduled by the Brain

- ROM layer: HookLogic-X writes observation into the system, breaking through commercial hardening
- Collaboration zone: a workspace for sharing artifacts and state, and a shared forum for dynamic coordination
- Hardware layer: test devices, the BLE research bench, and network equipment provide the physical foundation

IV. HookLogic-X: Writing Observation into the System

HookLogic-X performs system-level precompilation modifications based on AOSP 14 (a PixelBuilds derivative, kernel msm-4.14), modifying a total of 22 files and covering four levels: the kernel VFS layer, the ART runtime, Framework services, and the BoringSSL cryptography library. Its core design philosophy is to sink all capabilities into the ROM itself, making the attack surface part of the system itself.

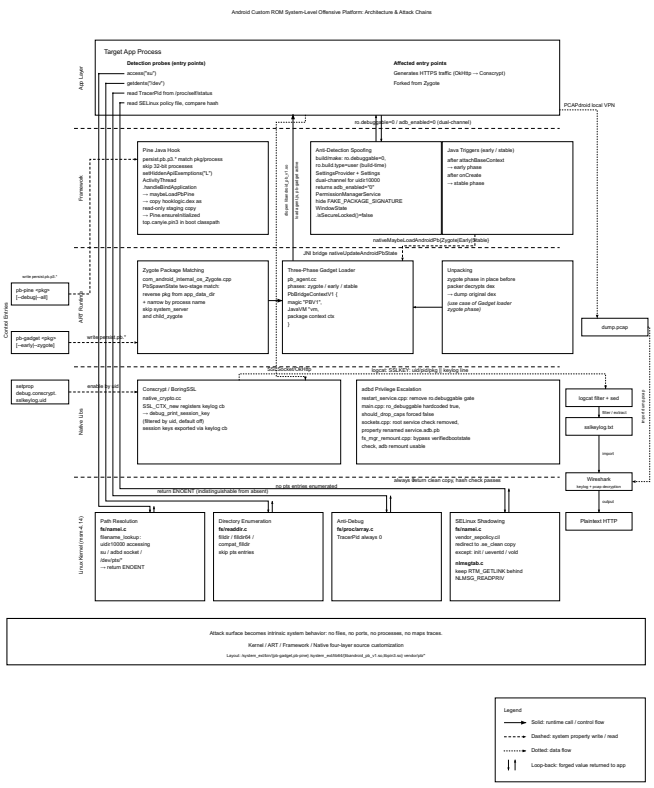


Fig. 2 Overall architecture and attack chain of the Android custom ROM system-level attack-defense platform.

A. Kernel VFS Source-Level Interception

Conventional Root hiding tools (such as Shamiko) intercept openat/stat through syscall hooks, leaving comparable signatures in kernel memory. HookLogic-X instead modifies logic directly at three original VFS callback points:

```
// fs/namei.c — interception at the path resolution
layer
static int filename_lookup(...) {
    if (is_hidden_path_name(name->name)) {
        putname(name);
        return -ENOENT;  // not EACCES, but "the file
truly does not exist"
    }
    ...
}
// fs/readdir.c — interception at the directory
enumeration layer (full coverage at
filldir/filldir64/compat_filldir)
if (current_uid().val >= 10000 && namlen >= 3
&& !strncmp(name, "pts", 3))
    return 0;  // skip this entry, do not write it
into the user-space buffer
// fs/proc/array.c — anti-debugging at the proc status
layer
if (current_uid().val >= 10000)
    seq_put_decimal_ull(m, "\nTracerPid:\t", 0);  //
always 0, even when being ptraced
```

Why the VFS source rather than syscall hooks: a hook itself leaves jump-instruction signatures in kernel memory, and multiple hook frameworks coexisting tend to trample on one another. Filtering at the source where VFS data flows out is equivalent to "the file truly does not exist"—ENOENT is completely indistinguishable from a genuine empty path, whereas EACCES proves the file exists.

B. Three-Stage Injector in the ART Runtime

The exposure surface of Frida Server (a standalone process, port 27042, thread names gmain/gum-js-loop) has already been catalogued by all hardening products. HookLogic-X compiles the Gadget loader directly into libart, loading it at three selectable timing points after Zygote forks an application process:

HookLogic-X exposes three hooking phases with different trigger points. In the zygote phase, hooks are installed at ZygoteHooks.postForkCommon(), before any application code executes and before the shell decrypts its dex, allowing the original in-memory dex to be dumped. In the early phase, hooks are placed after Application.attachBaseContext() and before onCreate(), covering the Application initialization path and enabling interception of MultiDex and dynamic loading. In the stable phase (the default), hooks are installed after the content providers and Application.onCreate(), specifically to avoid shells that initialize anti-debugging native libraries such as libmsaoaidsec and libkadp in onCreate(); the shell's self-check completes before the Gadget enters.

On the Zygote side, two-stage package-name matching is performed (first proving package ownership via app_data_dir, then narrowing to a single process), and injection is never performed into system_server or child_zygote.

C. Three-Stage Injector in the ART Runtime

The Java source of Pine (an ART-level Java method hooking framework) is compiled directly into the boot classpath. This means:

- The hook-logic dex does not need to bundle the Pine library itself, minimizing its size.

- Pine classes are "system classes" as far as the application process is concerned—the same category as framework classes—so a detector cannot determine anything from "the application loaded a suspicious class."
- It is complementary to the Gadget channel: Pine performs Java method-level hooking (argument/return-value tampering, call-stack tracing), while the Gadget performs native-layer hooking and scripted analysis.

Secure staging is performed at load time: rather than directly loading /data/user/0/<pkg>/pb/pine/hooklogic.dex (if it was pushed in via adb, its owner is shell/root, and the appearance within the target application process of "an executable dex in its own data directory that does not belong to it yet is writable by it" is an obvious IOC), the file is copied to the app-owned runtime/load.dex and set read-only, so that its file attributes are indistinguishable from the application's normal cache.

D. Conscrypt TLS Key Export

Android 7+ applications do not trust user CAs by default, and certificate pinning means that MITM packet capture yields nothing but handshake failures. Hooking SSL_read/SSL_write with Frida requires injecting into the process, and per-connection hooks suffer from both overhead and stability problems when facing a large number of connections.

HookLogic-X takes a fourth path: exporting TLS session keys from inside BoringSSL:

```
//
external/conscrypt/common/src/jni/main/cpp/conscrypt/n
ative_crypto.cc
static jlong NativeCrypto_SSL_CTX_new(JNIEnv* env,
jclass) {
    ...
    /* add-boluo: always register the keylog callback.
The callback internally decides whether to emit based
on a property,
       off by default, zero overhead. No longer
depends on the kWithJniTraceKeys compile-time switch.
*/
    SSL_CTX_set_keylog_callback(sslCtx.get(),
debug_print_session_key);
    ...
}
```

Inside the callback, targets are filtered by property (debug.conscrypt.sslkeylog.uid), and the output format is SSLKEY: uid=<uid> pid=<pid> pkg=<package name> || <NSS keylog line>; a single sed on the host side yields a clean keylog file, which Wireshark natively supports.

Since nearly all Java-layer TLS on Android converges on Conscrypt (including the Java binding paths of OkHttp, Retrofit, WebView, and Cronet), this single-point modification covers the HTTPS traffic of the vast majority of applications.

E. From "Able to Hook" to "Undetectable"

These four levels of modification allow HookLogic-X to establish a new capability boundary where conventional solutions fail.

Magisk's DenyList is essentially "hiding user space within user space," and can be bypassed by kernel-level or earlier-stage detection. HookLogic-X intercepts at the VFS source, so that when an application process accesses su, the adbd socket, or /dev/pts/* paths by any means, it uniformly receives ENOENT—completely indistinguishable from a genuine empty path. SELinux policy shadowing makes the policy files read by tools such as NativeTest have hashes identical to those of the official image, so that even the fact that "a different file is being opened" is invisible.

Conventional unpacking schemes (Xposed/Frida scripts dumping dex at the application layer) face anti-debugging shells that initialize during startup: the injection timing is too late, and the shell has already completed its self-check before the hook points are established. HookLogic-X's stable phase injects only after the shell's self-check has finished, and combined with kernel-level hiding (TracerPid always 0), the shell's runtime re-checks are likewise ineffective. Many hardened shells initialize anti-debugging natively in Application.onCreate(); the zygote phase would be discovered immediately, whereas the stable phase avoids this entirely.

MITM proxies are ineffective on Android 7+, which does not trust user CAs by default, and certificate pinning defeats them entirely. Frida hooking SSL requires injecting into the process, and per-connection hooking incurs high overhead and poor stability. HookLogic-X registers a keylog callback in Conscrypt's SSL_CTX_new, so that keys are exported inside the library while ciphertext is transmitted as usual, and offline decryption is performed in Wireshark in conjunction with passive capture (PCAPdroid's local VPN). Since OkHttp, Retrofit, and WebView all converge on Conscrypt, a single-point modification covers the vast majority of HTTPS traffic, and it is equally effective for QUIC/HTTP3 (the same SSL_CTX path).

Frida Server is a standalone process with a port, thread names, and traces in maps. HookLogic-X's Gadget loader is compiled into libart, so the act of loading is itself part of ART. Pine is compiled into the boot classpath, so after hooking there is no JNI bridge, no extra threads, and no suspicious .so appearing in maps. All attack surfaces change from "user-space attachments" to "inherent system behavior," and conventional detection methods based mainly on file signatures, port signatures, and process signatures largely fail.

F. System-Endogenous Observation

The common characteristic of these four levels is that the key modifications reside inside the custom system source code and the ART execution path, and that large-scale rewriting of the target application's method bodies is avoided in the first place.

From the perspective of the target App:

- No ptrace attachment (HookLogic-X is inside ART)
- No external .so mapping (the observation logic is compiled into boot.oat)
- No exposed control channel (data is exported via system properties/shared memory)
- Method bytecode is unmodified (entry replacement is preferred over inline patching)

- TracerPid in /proc/self/status is always 0 (falsified at the kernel proc layer)
- access("/system/bin/su", F_OK) returns ENOENT (intercepted at the VFS source)

This is not "a stealthier Frida," but rather making observation a behavior of the system itself. Just as a process cannot distinguish whether a given memory read is GC or a debugger, it cannot distinguish whether a given method link is ART's normal flow or a HookLogic-X observation point.

Advantage: commercially hardened Apps change from "black boxes" into "white boxes," and LLMs can directly read the real business logic. This is a system-level capability that application-layer solutions (Frida/Xposed) cannot achieve.

V. Three-Agent Architecture: Fusion, Isolation, and Mutual Exclusion

A. Architectural Design

AX adopts a three-agent architecture plus a toolkit layer:

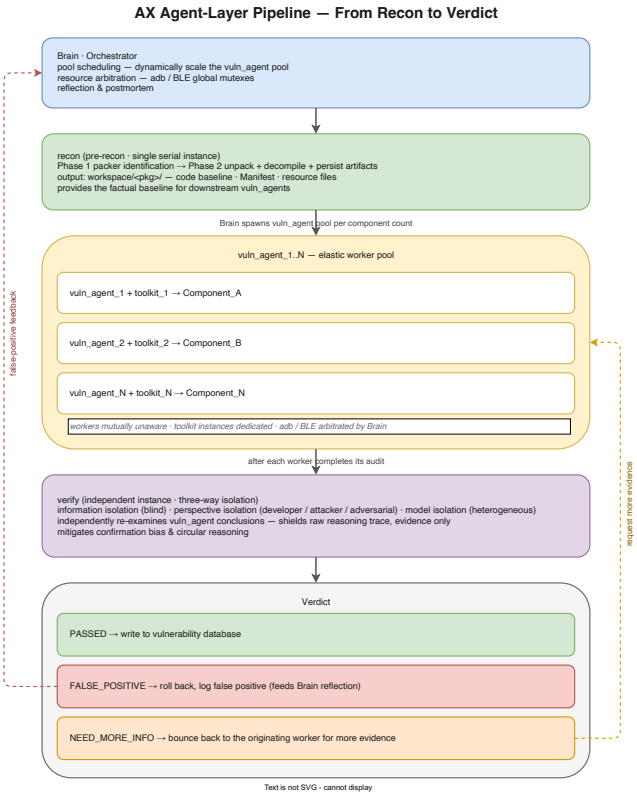


Fig. 3 Workflow of the AX Agent Layer: From Reconnaissance to Adjudication

Core roles:

- recon (front-end reconnaissance, single instance, serial): responsible for hardening identification → unpacking → decompilation → persisting original artifacts. Key discipline: produce only raw materials (code baseline/Manifest/resource files), without performing attack-surface mapping, API extraction, or IDOR scanning—avoiding cascading inheritance by subsequent workers of recon's erroneous conclusions, and avoiding worker cognitive fixation caused by preset conclusions.
- vuln_agent (elastic worker pool, 0~N): each worker independently handles the complete audit of one component (from reading code to producing a vulnerability report), maintaining cognitive closure.
- verify (an independent instance per component): independent adversarial review.
- toolkit (a paired instance per worker): four subtasks—cve_scan / traffic_audit / phone_automation / ble_control—serve as the worker's "hands." When a worker needs to scan CVEs, it calls toolkit.cve_scan; when it needs to capture traffic, it calls toolkit.traffic_audit; when it needs physical interaction, it calls toolkit.phone_automation or toolkit.ble_control.

B. Three-Layer Isolated Adversarial Review

verify is not a "parrot of the worker" but an adversarial reviewer:

Information isolation (blind-review principle): verify does not know the worker's reasoning process; it only examines the final evidence (PoC, traffic captures, code snippets). This avoids confirmation bias arising from "the worker's reasoning sounds plausible."

Perspective isolation (multi-perspective verification): verify reviews from multiple independent perspectives, with the core goal of preventing the misjudgment that "because one attack path failed, the vulnerability does not exist."

Key design: the "existence" of a vulnerability is decoupled from "the success of this particular path."

- Developer perspective: reasoning from the designer's standpoint, "what vulnerabilities would arise from the design pattern of this component?"—not judging whether the vulnerability the worker found exists, but proactively reasoning about all possible vulnerability types under that design pattern, and then checking whether the worker missed any. For example, on seeing WebView + JSBridge, the developer perspective reasons about risks such as "whitelist validation, interface permissions, token leakage," rather than merely re-reviewing the one path reported by the worker.
- Attacker perspective: apart from the attack path the worker found, what other attack angles have been ignored? If the worker's PoC fails, the failure eliminates only a single path, not the vulnerability hypothesis—the attacker perspective tries different exploitation methods (changing parameters, switching entry points, combining multiple weaknesses).
- Adversarial perspective: if I am told "this component has no problems," what counter-evidence can I find? "Missing evidence is not evidence of absence"—failure to find a PoC does not mean the vulnerability does not exist; it may be due to environmental constraints, insufficient privileges, or the need for specific trigger

conditions. The adversarial perspective checks: does the worker's "not exploitable" conclusion have sufficient evidence? Are there cases that "look like vulnerabilities but the worker did not try the right method"?

Key constraints: verify may not dismiss a vulnerability on the following grounds:

- "The worker's PoC failed" (this only shows that this path failed, not that the vulnerability does not exist)
- "A secure sibling call site exists" (A being secure does not mean B is secure)
- "The control is on another path" (middleware protects the main route but not internal callers)
- "I cannot reproduce it" (environmental/privilege issues do not mean the vulnerability does not exist)

Three-state conclusions:

- CONFIRMED: a working PoC exists, or a complete source → control → sink → impact chain plus reachability evidence
- RULED_OUT: it must be possible to state explicitly "this component is secure because <specific control> does <what> at <file:line>, and it takes effect on every path reachable by the attacker"
- OPEN_PROOF_GAP: "can neither be confirmed nor ruled out" is a legitimate and expected terminal state—recorded as needs_follow_up and handed over to subsequent audits or manual review

Advantage: the standard shifts from "if the worker says there is none, there is none" to "verify must prove there is none," greatly reducing the false-negative rate.

Model isolation (heterogeneous adversarial): the worker uses model A (e.g., glm-5.3), and verify uses model B (e.g., deepseek-v4.1flash)—different vendors, different architectures, different training data. In case of conflict, a third model C (e.g., kimi-k3) is introduced for arbitration. Research at ICLR 2026 (arXiv:2509.17995) demonstrates that verifiers and generators from the same family share blind spots, and that heterogeneous models are key to suppressing consensus traps.

VI. The Collaboration Layer: Workspace + Shared Forum

All agents collaborate through the Collaboration Layer—the infrastructure beneath the Brain, comprising two complementary mechanisms.

A. *Blackboard: A State-Driven Asynchronous Collaboration Hub*

workspace/<pkg>/ is the single source of truth, and the directory structure itself constitutes the coverage ledger:

```
workspace/<pkg>/
├── recon/
│   ├── manifest.xml         # artifact
│   ├── code_baseline/       # artifact
│   └── status.json       # state: { "phase": "co
mpleted", "evidence": "..." }
├── components/
│   ├── MallWebActivity/
│   │   ├── vuln_report.md   # artifact
│   │   ├── poc.har          # artifact
│   │   └── status.json      # state: { "verdict
": "confirmed", "confidence": "HIGH" }
│   ├── BluetoothService/
│   │   └── status.json      # state: { "verdict
": "pending", "missing_evidence": ["BLE connectio
n required"] }
│   └── ...
└── forum/
    ├── thread_001.md        # shared forum messa
ge
    └── ...
```

Roles of the blackboard:

- State-driven: Agents monitor changes to specific fields on the blackboard and act autonomously when the state satisfies a trigger condition. When recon writes code_baseline/, the vuln_agent's listener is immediately triggered and auditing begins; when vuln_agent writes verdict: confirmed to status.json, the verify listener is immediately triggered and review begins. Without explicit scheduling by the Brain, agents decide autonomously whether to act.
- Token savings: Each agent subscribes only to the fields it cares about (e.g., vuln_agent subscribes only to the verdict field of components/*/status.json) and does not need to load the full message history. Compared with an orchestrator-worker architecture, the blackboard pattern saves approximately 59% of token consumption.
- Fault-domain isolation: If a worker crashes, other workers can recover the scene from the blackboard and

continue working. The blackboard is the sole persistent store; even if the Brain restarts, no state is lost.

- Knowledge accumulation: The blackboard is the sole persistent store across Apps and audit cycles, and a temporal knowledge graph extracts vulnerability patterns from it (20+ categories such as whitelist bypass, JSBridge leakage, and BLE protocol flaws).
- The status.json in each component directory is the coverage ledger entry:
- confirmed: a working PoC exists, or a complete source → control → sink → impact chain plus reachability evidence
- pending: the missing evidence must be listed ("real account required," "BLE connection required," "root privilege required")
- refuted: it must be possible to state explicitly "this component is secure because <specific control> does <what> at <file:line>, and it takes effect on every path reachable by the attacker"
- not_applicable: the reason for non-applicability must be stated ("this component is not exported," "this protocol is not enabled")
- needs_follow_up: "can neither be confirmed nor ruled out" is a legitimate and expected terminal state, handed over to subsequent audits or manual review

Key constraint: "Missing evidence is not evidence of absence"—failure to find a PoC does not mean the vulnerability does not exist; it may be due to environmental constraints, insufficient privileges, or the need for specific trigger conditions.

B. Shared Forum: Dynamic Communication (Message-Level)

The dynamic coordination mechanism among agents follows the principle of contractual tool invocation (agents treat peers as "tools with well-defined inputs and outputs," rather than "long-term peers without hierarchy"):

- Lead broadcasting: when vuln_agent_1 finds that "this App's WebView whitelist uses contains matching," it writes to forum/thread_001.md, and other workers can check whether the components they are responsible for exhibit the same pattern.
- Help coordination: when vuln_agent_2 needs BLE protocol analysis capability but lacks ble_control permission, it posts a request on the forum, and the Brain coordinates a worker or toolkit instance with the appropriate permission to assist.
- Counter-evidence sharing: when verify finds that a certain class of vulnerability is a false positive, it broadcasts the counter-evidence pattern ("this kind of whitelist check appears bypassable, but there is actually a second layer of validation"), preventing other workers from repeating the same mistake.

Forum messages are stored in the workspace/<pkg>/forum/ directory, sharing the same storage as the workspace artifacts, with no need for a separate message queue.

Relationship to the graded reporting protocol: the graded reporting protocol is upward reporting (Agent → Brain), whereas the shared forum is lateral collaboration (Agent ↔ Agent). Both are realized through the collaboration layer—graded reporting writes to the component directory's status.json, and forum messages are written to the forum/ directory.

Graded reporting protocol (all agents must comply, to avoid "misunderstandings caused by free-form output"):

- recon must report Phase/Status/Evidence/Blocker
- vuln_agent must report Component/Confidence/Evidence/Limits
- verify must report Verdict/Confidence/Counter_Evidence

Confidence is the only mandatory cross-role field — confidence propagates along the recon → vuln_agent → verify chain, and the Brain can make resource-scheduling decisions based on confidence.

In addition, AX accumulates vulnerability patterns across Apps and audit cycles through a temporal knowledge graph (20+ categories such as whitelist bypass, JSBridge leakage, and BLE protocol flaws), and ensures knowledge purity through a verification-driven distillation pipeline—only conclusions that pass verify may enter long-term memory (Playbook), and FALSE_POSITIVE cases are recorded as anti-patterns.

Advantages:

- Comprehensive coverage: the workspace is the ledger, and the audit status of every component is traceable, avoiding "missing an exported activity"
- Accurate reporting: three-state adjudication (confirmed/pending/refuted) makes the evidence chain of every conclusion complete and auditable
- Good collaboration: the shared forum makes emergent collaboration among agents possible (Anthropic 2026 experiment: collaborative swarm vulnerability output is 12 times that of independent parallel work)

VII. THE COLLABORATION LAYER: WORKSPACE + SHARED FORUM

The BLE research bench is the core infrastructure of AX's software-hardware integration, providing the physical-layer protocol analysis capability for toolkit.ble_control. We designed a dual-track hardware solution: the commercial

deployment is used for in-depth internal research, while the personal deployment allows Bluetooth security enthusiasts to reproduce our attack chains as well.

A. Commercial Deployment: Ellisys Capture + nRF52840 Transmission

Ellisys Bluetooth Vanguard (capture side):

a) Hardware specifications: a professional-grade BLE protocol analyzer supporting simultaneous sniffing of all BLE 5.3 channels (37/38/39), with hardware timestamp accuracy of ±1 μs and a built-in large-capacity capture buffer.

b) Core capabilities:

- Physical-layer full capture: not constrained by connection state; can capture raw radio frames of Advertising, CONNECT_IND, and Data Channel PDUs—including CRC-error frames, retransmitted frames, timing deviations, and other low-level details essential for protocol reverse engineering.
- Protocol reverse engineering: captures the complete connection establishment process (CONNECT_IND → LL_FEATURE_REQ → LL_ENC_REQ → ...), reconstructing the handshake sequence of vendor-proprietary protocols.
- Timing analysis: precisely measures the transmission time of each packet and detects timing side channels (e.g., some vehicles respond with a delay upon receiving an incorrect key).
- Multi-protocol same-screen view: BLE / Classic / Wi-Fi are time-aligned on the same screen, facilitating cross-protocol timing analysis (e.g., the causal chain between a BLE unlock command and vehicle-side CAN wake-up).
- Interference-resilience verification: verifies the robustness of the attack chain under packet loss/jitter in complex electromagnetic environments (parking lots, concurrent devices in 4S dealerships).

c) Data output: .btt capture files are automatically converted to PCAPNG and written to the blackboard at workspace/<pkg>/ble_captures/, for offline replay during verify's review.

nRF52840 Dongle (transmit side):

a) Hardware specifications: Nordic nRF52840 SoC (ARM Cortex-M4F @ 64 MHz, 1 MB Flash, 256 KB RAM), supporting the full BLE 5.0 protocol stack (Advertising/Scanning/Central/Peripheral).

b) Firmware: flashed with Nordic's official NCS (nRF Connect SDK) hci_usb firmware, exposing it to the host as a standard USB Bluetooth controller through the HCI interface.

c) Core capabilities:

- Active attack: acts as a Central to connect directly to the vehicle and send custom GATT writes; acts as a Peripheral to clone the vehicle's advertising and lure the phone App into actively connecting.
- Dual-role relay: two Dongles—one as a Central connecting to the vehicle, one as a Peripheral connecting to the phone—with Bumble forwarding PDUs between them, achieving an end-to-end relay attack.
- Address spoofing: the MAC address can be modified, bypassing simple identity binding based on MAC whitelists.
- Cost advantage: ~$10 per unit, making it the standard entry-level device for Bluetooth security research.

Why this division of labor: Ellisys provides the radio-frequency perspective (raw over-the-air frames, including physical-layer information such as retransmissions, timing, and signal strength), whereas the nRF52840 provides the protocol-stack perspective (PDUs already processed by the Link Layer). Ellisys is responsible for "seeing clearly," and the nRF52840 for "striking accurately"—only by combining the two can one verify both that the attack is reachable at the physical layer and that the protocol-layer semantics are correctly parsed.

B. Personal Deployment: Dual nRF52840 Capture + Transmission

Hardware specifications: two Nordic nRF52840 Dongles, ~$10 each, the standard entry-level device for Bluetooth security enthusiasts; together with a laptop, the entire attack chain can be completed.

Role assignment:

- nRF52840 #1 (capture/vehicle side): acts as an Observer listening to target advertising; during the attack phase it switches to Central to connect directly to the vehicle.
- nRF52840 #2 (phone-luring side): acts as a Peripheral cloning the target vehicle's advertising, luring the victim's phone App into actively connecting.
- Relay attack: one Dongle connects to the vehicle and the other to the phone, with Bumble forwarding PDUs in between—achieving an end-to-end bidirectional relay.

Google Bumble protocol stack (software layer):

What it is: Google's open-source pure-Python BLE protocol stack, with complete implementations of protocol layers such as Link Layer / L2CAP / ATT / GATT / SMP.

Why not use the host operating system's Bluetooth stack: the Bluetooth stacks of Android/iOS are black boxes, incapable of precisely controlling low-level behavior (e.g., custom advertising intervals, modifying PDU fields, adjusting Connection Parameters, forcing downgrade of pairing parameters). Bumble is a pure-software implementation with full control over every protocol field (advertising payload, PDU header, ATT opcode, SMP pairing parameters)—this is the prerequisite for protocol attacks.

Core capabilities:

- Scripted driving: toolkit.ble_control precisely controls the nRF52840's role switching (Observer ↔ Central ↔ Peripheral) through Bumble's Python API, scheduling advertising/scanning/connection timing with millisecond-level precision.
- Protocol-field-level fuzzing: constructs malformed PDUs (illegal ATT_MTU, out-of-bounds GATT handle, incorrect L2CAP length field) to test the target device's protocol-parsing robustness.
- Custom pairing flow: bypasses the standard SMP flow and manually controls every field of Pairing Request/Response (forcing lower IO Capability to trigger Just Works pairing, modifying Encryption Key Size to trigger downgrade negotiation)—a key capability for verifying KNOB-class negotiation downgrade flaws (CVE-2019-9506).
- Cross-platform consistency: the same set of attack scripts behaves identically on Android Termux, Linux USB, and macOS—verifying the protocol-layer essence of the attack chain, decoupled from the host operating system.

Key design: AX's BLE attack chain is fully decoupled from the radio/host stack—the same attack code can run on Ellisys + nRF52840 (commercial deployment), on dual nRF52840 (personal deployment), or even on Linux BlueZ (pure software). This verifies the protocol-layer essence of the attack chain, which does not depend on specific hardware.

C. *Hardware Collaborative Workflow*

Taking "BLE digital key attack" as an example, this illustrates how three types of hardware (Ellisys / nRF52840 / the Bumble protocol stack) collaborate. The shared reconnaissance phase produces three types of intelligence, which feed three parallel attack paths—if any one of them succeeds, the vulnerability is confirmed:

(The complete workflows of the three paths are shown in Figure 4.)

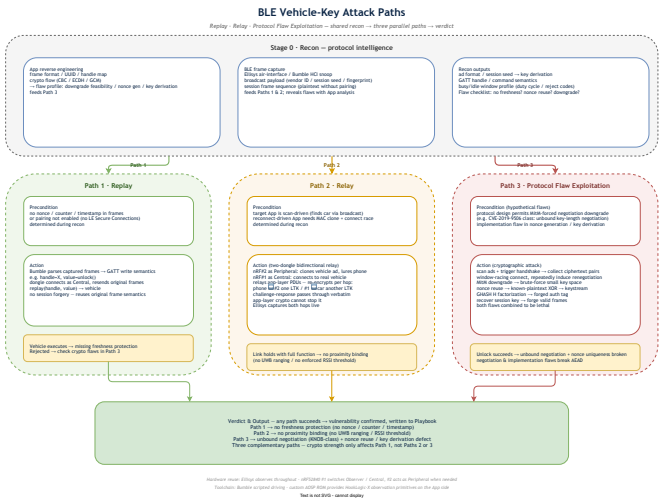


Fig. 4 Three Types of Attack Paths Against BLE Digital Keys: Replay / Relay / Protocol Flaw Exploitation

D. *Relationship with toolkit.ble_control*

- toolkit.ble_control is the software abstraction layer: when vuln_agent calls toolkit.ble_control.scan(), the underlying operation is Bumble controlling nRF52840 #1 to enter Observer mode.
- Hardware conflicts are self-handled: when multiple workers operate the BLE hardware simultaneously, toolkit.ble_control itself decides whether to queue and retry or switch to an idle Dongle.
- Ellisys data is automatically synchronized: Ellisys Vanguard's capture files (.btt format) are automatically converted to PCAPNG and written to the blackboard at workspace/<pkg>/ble_captures/, for verify's review.

Design constraint: occupancy conflicts over the BLE antennas (two nRF52840s + Ellisys) are handled by toolkit.ble_control itself (queue and retry / switch Dongle / report an error); the Brain does not perform resource arbitration.

VIII. Vulnerability Results

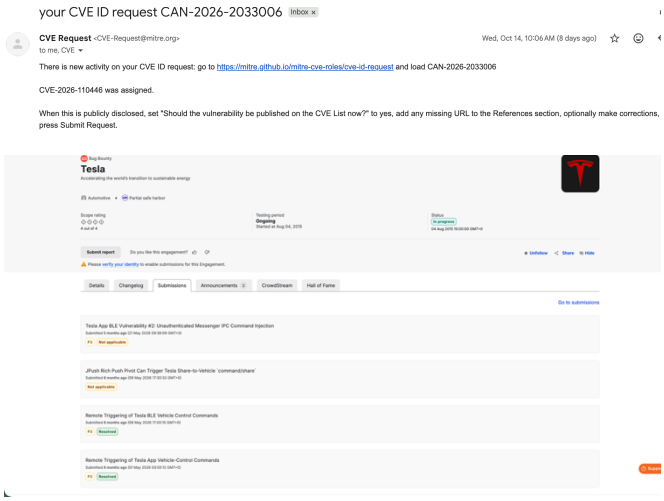
AX has completed large-scale validation in authorized security research across multiple leading automakers, covering real-vehicle attack scenarios.

A. *Tesla: 3 Vulnerabilities, 1 CVE*

- CVE-2026-110446: zero-permission vehicle control. A zero-permission App can directly control the vehicle across the entire lineup, with the owner completely unaware. Tesla has acknowledged the issue and completed a security fix, and the finding received a Bugcrowd bounty of USD 1,500.
- Imperceptible navigation tampering. A second finding, not yet assigned an identifier, allows imperceptible tampering with the in-vehicle navigation destination. The vendor declined to fix it as "intended design

behavior"; the complete attack chain will be publicly released on site.

- Duplicate report. The third finding was deemed by the vendor to be a duplicate of an existing external report and did not receive a separate identifier.



B. Other CVEs and CAPPVD Identifiers

CVE-2026-7516, CVE-2026-7554: cloud component vulnerabilities.

22 CAPPVD identifiers (the Chinese counterpart of CVE, a national-level vulnerability database under the MIIT system), covering multiple leading automakers and mobility platforms: NI*,FAW-**,SA** Motor, etc. (vendor names have been anonymized).

C. BLE Digital Key Weaknesses: The Complete App–Vehicle Linkage Chain

HookLogic-X broke through the hardening, and vuln_agent obtained the complete business logic of the BLE digital key. toolkit.ble_control performed protocol analysis next to a real vehicle and discovered multiple protocol-layer weaknesses, enabling remote vehicle control without the victim's account.

verify's independent-model review confirmed that these are not false positives. They have been independently reproduced on three platforms—Android BLE (nRF52840), Linux BlueZ, and macOS—demonstrating that the attack chain is fully decoupled from the radio/host stack.

(Details are confidential due to the coordinated disclosure period; the complete attack chain will be presented on site at the conference.)

IX. Related Work and Differentiation

A. AI Agent Offense and Defense Research in 2026

The GPT Agent autonomous attack incident disclosed by OpenAI at Black Hat USA 2026, GitHub Security Lab's seclab-taskflows (24 Android vulnerabilities), and Zhiqiang Peng's annual report at OffByOne 2026 (nearly 1,000 vulnerabilities discovered by LLM + Agent on custom Android systems) collectively mark the transition of AI Agents from auxiliary tools to autonomous entities capable of independently completing vulnerability research.

The essential difference between AX and these works is that they assume the target is observable by existing tools, whereas AX, through HookLogic-X, turns observation from an adversarial problem into a system-capability problem..

B. Multi-Agent Collaboration and Hallucination Suppression

The Fudan team's AgentFuzz discovered 34 0days (including 23 CVEs) through an "attack Agent versus target Agent" framework; PentestAgent and Strix integrate multi-agent plus toolchains into penetration testing scenarios; GitHub seclab-taskflows proposes "decomposing research into incremental steps" to enhance LLM vulnerability discovery; VEXAIoT introduces a multi-agent framework into IoT exploitation.

The difference between AX and these works is that AX, through its three-agent architecture plus three-layer isolated adversarial review, suppresses hallucinations and consensus traps on long-chain tasks, reducing the false-positive rate from 40% to below 5%.

C. Application-Layer Dynamic Instrumentation and Automated Unpacking

Frida and Xposed have long been the de facto standard for mobile dynamic analysis, but the mechanisms they rely on—ptrace, external .so mapping, control channels—have themselves become detection signatures in the face of modern hardening. Representative work such as FART was the first to propose intercepting the ClassLinker path to recover the restored CodeItem.

HookLogic-X follows this idea but upgrades it from a single-point tool to a system capability, encapsulating it together with method-level hooking, key export, and device-fingerprint spoofing primitives in the same substrate.

D. Summary of Four Divergences

- Route. Mainstream work optimizes the agent itself—better prompts, more tools, longer context—whereas AX is observation-first: it builds observation capability into the system rather than treating it as something the agent must acquire through tools.
- Capability boundary. In mainstream practice, root hiding, injection frameworks, hooking frameworks, and packet-capture tools each exist as separate point tools; AX instead integrates kernel interception, runtime injection, framework-layer hooking, and TLS key export into a single system-native substrate.
- Adversarial mechanism. Mainstream work relies on a single model or multiple models from the same family, while AX enforces three-layer isolation across information, perspective, and model, and further decouples vulnerability existence from exploit-path success.
- Attack surface. Mainstream work audits the App, the BLE protocol, or the cloud in isolation; AX covers all

three end-to-end, traversing App source code, cloud interfaces, BLE protocol, and finally the physical vehicle.

X. Conclusion

In 2026, AI Agents have moved from concept demonstrations to real-world offense and defense, yet almost all public work sidesteps the same question: when the target is wrapped layer upon layer in commercial-grade hardening, what can AI still see clearly?

AX's answer is: do not stuff AI into old tools; put AI on a substrate that can see the truth.

HookLogic-X makes observation a system capability, the three-agent architecture makes AI dig accurately, and the software-hardware integration extends the attack surface from the App layer to the physical layer. 3 CVEs (including the Tesla full-lineup imperceptible control CVE-2026-110446), 22 CAPPVD identifiers, and three attack paths against the BLE digital key—these are not the endpoint but the starting point. When observation becomes a system capability and AI is no longer blind, the rules of the game for Internet of Vehicles security will be thoroughly rewritten.

This represents a paradigm shift in mobile security auditing in the AI era, from "expert craftsmanship" to "system-endogenous engineering."



Fig. 5 live demonstration

Acknowledgment

We thank the security response teams of all affected vendors for their professional handling of the vulnerability reports, and we thank open-source projects such as AOSP, Google Bumble, the Zephyr Project, Nordic NCS, and Pine, as well as the conference communities of Black Hat, USENIX Security, and OffByOne for their continued advancement of AI Agent security.